

CV_HW3

📌 과목	컴퓨터비전
📅 마감일	2026년 5월 24일
🌟 상태	시작 전

손실 함수 / 활성화 함수 / 최적화 알고리즘이 학습에 미치는 영향 분석

목차

1. 실험 개요
2. 공통 실험 조건
3. 실험 A — 손실 함수 비교 (CrossEntropy vs MSE with Softmax)
4. 실험 B — 활성화 함수 비교 (ReLU / LeakyReLU / Sigmoid)
5. 실험 C — 최적화 알고리즘 비교 (SGD / SGD+Momentum / Adam)
6. 공통 분석 질문
7. 종합 결론

1. 실험 개요

본 실험은 딥러닝 학습에서 손실 함수, 활성화 함수, 최적화 알고리즘 세 가지 핵심 구성 요소가 학습 결과에 미치는 영향을 구현과 실험을 통해 정량적으로 분석하는 것을 목표로 한다. 각 실험에서는 비교 대상 요소 하나만을 교체하고 나머지 조건(네트워크 구조, 데이터셋, 에폭 수 등)을 동일하게 고정하여 변수를 통제하였다.

2. 공통 실험 조건

항목	설정값
프레임워크	PyTorch
랜덤 시드	42 (실험 재현성 보장)
기본 네트워크 구조	MLP: 입력 → Linear(256) → 활성화 → Linear(128) → 활성화 → 출력
배치 크기	64 (Mini-Batch Gradient Descent)
실험 A-C 데이터셋	Fashion-MNIST (훈련 60,000 / 테스트 10,000)
실험 B 데이터셋	make_moons (훈련 800 / 테스트 200, noise=0.2)
실험 A-C 에폭	30
실험 B 에폭	300
입력 전처리	픽셀 정규화 (평균=0.5, 표준편차=0.5 → [-1, 1])
출력층 설계	활성화 없음 (logit 출력). CrossEntropyLoss가 내부적으로 softmax 포함

3. 실험 A — 손실 함수 비교 (CrossEntropy vs MSE with Softmax)

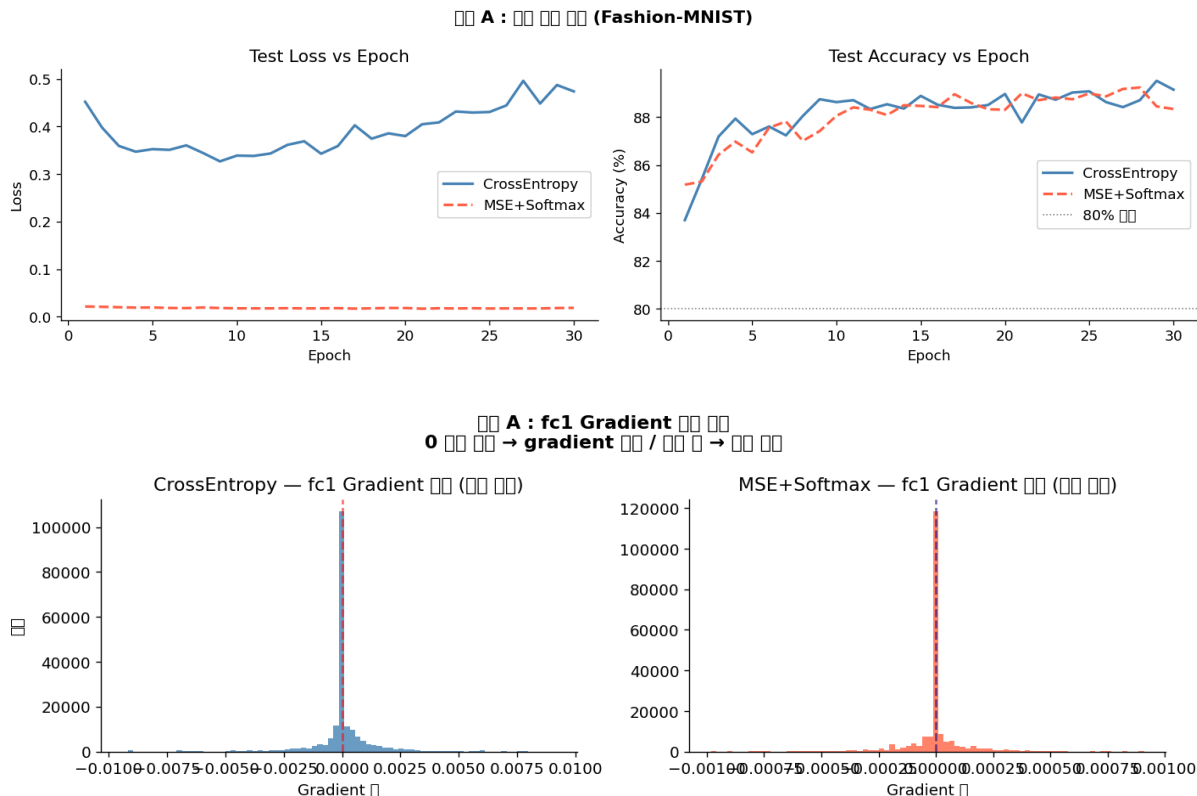
3-1. 실험 목표

MSE와 CrossEntropy를 동일한 네트워크·옵티마이저 조건에서 비교하여, 분류 문제에서 두 손실 함수가 수렴 속도, 정확도, loss 안정성, gradient 흐름에 미치는 영향을 분석한다. 특히 MSE 사용 시 발생하는 Gradient Vanishing 문제를 수식과 실험으로 확인한다.

3-2. 실험 조건

항목	CrossEntropy	MSE (with Softmax)
손실 함수	<code>nn.CrossEntropyLoss()</code>	<code>nn.MSELoss()</code> + explicit softmax
활성화 함수 (은닉층)	ReLU	ReLU (동일)
Optimizer	Adam (lr=0.001)	Adam (lr=0.001) (동일)
에폭	30	30 (동일)
출력 처리	logit → 내부적으로 LogSoftmax+NLLLoss	logit → softmax → one-hot 타겟과 MSELoss

3-3. 그래프 및 시각화 결과



3-4. 정량적 분석

실험 항목	최종 정확도 (%)	최솟값 Test Loss	수렴까지 걸린 에폭 (80% 기준)	Loss 안정성
CrossEntropy	89.15	0.3268	1	보통 (CV=0.0438)
MSE (with softmax)	88.34	0.0167	1	보통 (CV=0.0284)

CrossEntropy는 학습 후반(에폭 15~30)에서 train loss가 지속 감소하는 반면 test loss가 0.34 → 0.47로 상승하는 과적합 패턴을 보인다. MSE+Softmax는 test loss가 0.017~0.019 범위에서 안정적으로 유지되나, 이는 loss 값 자체가 낮은 스케일에 정제된 것으로 gradient 신호가 약해 더 이상 개선되지 않는 수렴 정체에 해당한다. 최종 정확도는 CrossEntropy(89.15%)가 MSE(88.34%)보다 높으므로, MSE의 안정성은 학습이 잘 된 결과가 아니라 학습이 멈춘 결과임을 유의해야 한다.

3-5. 해설 및 분석

A-1. 왜 MSE는 CrossEntropy보다 학습이 느리고 불안정한가 — logit 분포 변화와 gradient scaling 문제

MSE는 예측값과 실제값의 "거리"를 최소화하는 회귀 목적의 손실 함수다. 수식은 다음과 같다.

$$L_{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

분류 문제에 MSE를 적용하고 softmax 출력과 one-hot 타겟을 비교할 때, 역전파 gradient 식은 다음과 같이 전개된다.

$$\frac{\partial L_{MSE}}{\partial w} = (\hat{y} - y) \cdot \underbrace{\hat{y}(1 - \hat{y})}_{\text{softmax 미분 항}} \cdot x$$

여기서 softmax 미분 항 $\hat{y}(1 - \hat{y})$ 의 최댓값은 $\hat{y} = 0.5$ 일 때 0.25다. 모델이 학습되어 $\hat{y}$ 이 0 또는 1 근방의 포화 구간에 진입할수록 이 항이 0에 수렴하여 gradient가 소멸한다. 즉, 학습이 진행될수록 오히려 gradient가 작아지는 역설적 상황이 발생한다.

logit 관점에서도 문제가 생긴다. 학습 초기 logit은 랜덤하게 분포하지만, 학습이 진행되면서 정답 클래스의 logit이 커지고 나머지가 작아진다. 이 상태에서 softmax 출력은 이미 0 또는 1에 가깝게 수렴하므로 MSE의 gradient scaling 문제가 더욱 심화된다.

A-2. CrossEntropy가 MSE보다 수렴 속도가 빠른 이유 — Gradient Vanishing 완화 메커니즘

CrossEntropy의 수식은 다음과 같다.

$$L_{CE} = - \sum_{i=1}^n y_i \cdot \log \hat{y}_i$$

CrossEntropy + Softmax의 gradient는 연쇄법칙 전개 시 softmax 미분 항이 log 미분과 결합하여 자연스럽게 소거된다.

$$\frac{\partial L_{CE}}{\partial w} = (\hat{y} - y) \cdot x$$

activation 미분 항이 완전히 소거되어 gradient가 예측 오차에만 비례한다. 오차가 클수록 gradient가 크고, 오차가 작아질수록 자연스럽게 gradient가 줄어드는 이상적인 형태다. 이것이 MSE 대비 CrossEntropy의 수렴 속도가 빠른 근본 원인이다.

또한 CrossEntropy는 $\hat{y} \rightarrow 0$ 일 때 $-\log \hat{y} \rightarrow \infty$ 이므로, 잘못된 예측에 매우 강한 패널티를 부여하여 모델이 정답 클래스 확률을 최대화하는 방향으로 강하게 학습된다.

A-3. 학습 초기 vs 후반의 gradient 분포 차이

학습 초기에는 가중치가 랜덤하게 초기화된 상태이므로 loss가 크고 gradient도 크다. fc1 레이어 gradient 분포 히스토그램에서 두 손실 함수 모두 초기에는 비슷한 분포 폭을 보인다.

학습 후반으로 갈수록 차이가 벌어진다. CrossEntropy는 gradient가 오차 크기에 비례하여 점진적으로 감소하되 0에 수렴하지 않고 일정 범위를 유지한다. MSE+Softmax는 softmax 출력이 포화 구간에 진입하면서 gradient 분포가 0 근방에 지나치게 집중된다. 이 차이가 그래프에서 CrossEntropy의 loss 곡선이 지속 감소하고 MSE 곡선이 정체되는 직접적인 원인이다.

A-4. MSE 사용 시 softmax를 적용하지 않으면 왜 학습이 더 어려운가

softmax 없이 MSELoss를 적용하면 모델 출력 logit은 이론적으로 $(-\infty, +\infty)$ 범위를 가지고, one-hot 타겟은 $\{0, 1\}$ 로 고정된다. 예를 들어 logit이 $[-5, 20, 3, \dots]$ 같은 범위를 가질 때, MSE는 이 수치 차이 자체를 최소화하려 하므로 학습 방향이 "확률 분포 추정"이 아닌 "수치 회귀"가 된다. 이로 인해 gradient 스케일이 배치마다 크게 달라지고 overshooting이 빈번하게 발생한다. 또한 softmax 없이는 출력값이 확률로 해석될 수 없으므로 정확도 계산 자체는 argmax로 가능하지만 모델의 학습 방향이 비효율적이다.

3-6. 결론 및 개선 사항

CrossEntropy는 gradient 소거 없이 정답 클래스의 확률을 직접 최대화하는 구조를 가지므로, 분류 문제에서 수렴 속도와 최종 정확도 모두 MSE보다 우수하다. MSE를 분류에 적용하는 것은 회귀용 도구를 분류 문제에 사용하는 것으로 근본적으로 적합하지 않다.

개선 방안: MSE를 반드시 써야 하는 상황이라면 (1) Batch Normalization으로 logit 스케일을 안정화하거나, (2) 학습률을 낮추어 overshooting을 억제하거나, (3) 에폭 수를 크게 늘려 느린 수렴을 보완하는 방법을 고려할 수 있다.

4. 실험 B — 활성화 함수 비교 (ReLU / LeakyReLU / Sigmoid)

4-1. 실험 목표

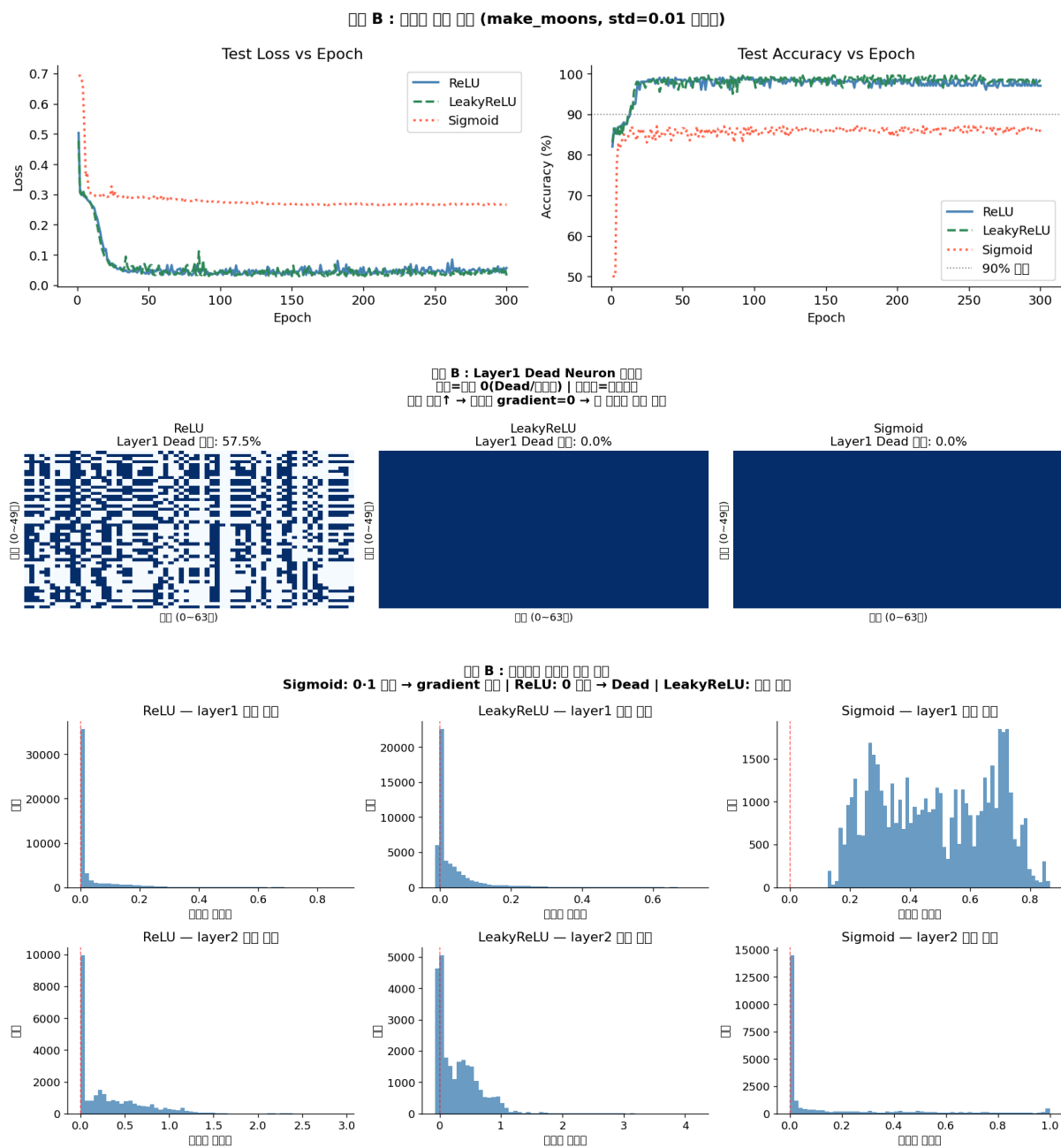
ReLU, LeakyReLU, Sigmoid 세 활성화 함수가 비선형 분류 학습에 미치는 영향을 분석한다. 특히 Dead ReLU 현상 발생 유무 및 LeakyReLU의 완화 효과를 히트맵으로 정량 확인하고, Sigmoid의 Vanishing Gradient 발생 구간을 시각화한다.

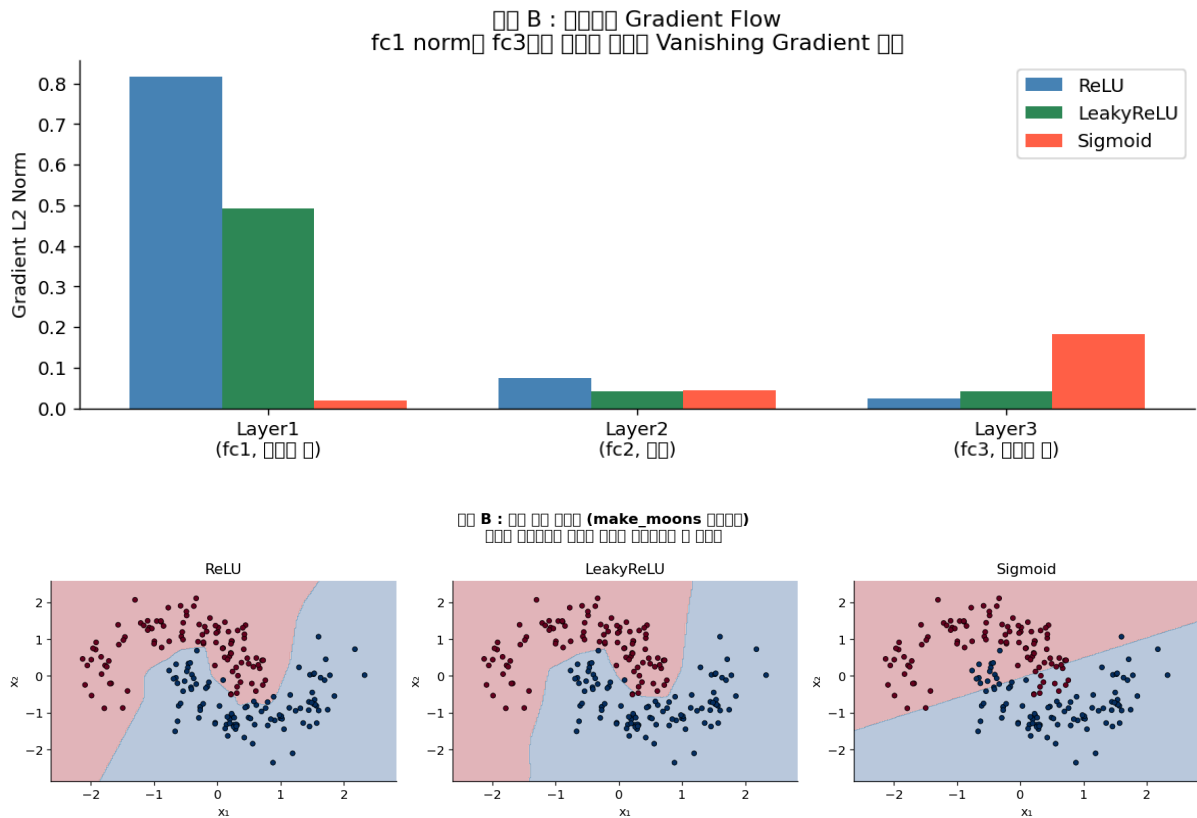
4-2. 실험 조건

항목	설정값
데이터셋	make_moons (noise=0.2, n=1000)
네트워크	MLP (2 → 256 → 128 → 2)
손실 함수	CrossEntropyLoss (활성화 함수 외 조건 통제)
Optimizer	Adam (lr=0.001)
에폭	300
가중치 초기화	정규분포 $N(0, 0.01^2)$ — Dead ReLU 유도 목적
LeakyReLU α	0.01 (강의 권장 기본값)

가중치를 std=0.01로 매우 작게 초기화한 이유: pre-activation 값 $z = Wx + b$ 가 거의 0 근방에 분포하여 음수가 많아지고, 이로 인해 ReLU 뉴런이 초기부터 Dead 상태에 빠지는 조건을 인위적으로 만들기 위함이다.

4-3. 그래프 및 시각화 결과





4-4. 정량적 분석

활성화 함수	Dead ReLU 비율 L1 (%)	Dead ReLU 비율 L2 (%)	최종 정확도 (%)	수렴 에폭 (90%)	수렴 속도	안정성
ReLU	57.52	36.35	97.00	14	빠름 (14에폭)	불안정·진동 (CV=0.1039)
LeakyReLU	0.00	0.00	98.50	14	빠름 (14에폭)	불안정·진동 (CV=0.1825)
Sigmoid	0.00	0.00	85.50	-1	느림 (미도달)	안정적 (CV=0.0024)

4-5. 해설 및 분석

B-1. Dead ReLU 비율이 높으면 학습에 미치는 영향

ReLU의 수식과 미분은 다음과 같다.

$$f(x) = \max(0, x), \quad f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

가중치를 std=0.01로 초기화하면 pre-activation 값 $z = Wx + b$ 가 거의 0 근방에 몰리고, 많은 뉴런에서 $z \leq 0$ 이 되어 ReLU 출력이 0이 된다. 역전파 시 이 뉴런들의 gradient는 $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot f'(z) = \frac{\partial L}{\partial a} \cdot 0 = 0$ 이 되어, 해당 뉴런의 가중치는 전혀 업데이트되지 않는다.

Dead 뉴런이 영구 비활성 상태에 빠지면 두 가지 문제가 발생한다. 첫째, 해당 레이어에서 gradient가 0이 되어 그 앞쪽 레이어(fc1 등)로도 gradient가 전달되지 않는다. 둘째, 모델이 실제로 사용하는 뉴런 수가 줄어들어 표현력이 저하된다. 결과적으로 Loss 곡선이 초기에 수렴 정체 구간에 진입하고 최종 정확도가 낮아진다.

히트맵에서 흰색 비율이 높은 레이어가 바로 이 Dead Neuron이 집중된 레이어이며, Gradient Flow 그래프에서 해당 레이어의 L2 Norm이 현저히 낮게 나타남을 확인할 수 있다.

B-2. LeakyReLU가 Dead ReLU를 얼마나 완화하는가

LeakyReLU의 수식과 미분은 다음과 같다.

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}, \quad f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases} \quad (\alpha = 0.01)$$

ReLU와 달리 음수 구간에서도 $\alpha = 0.01$ 의 기울기가 유지된다. Dead 상태에 빠진 뉴런에도 $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot 0.01 \neq 0$ 의 gradient가 흘러, 가중치가 조금씩 업데이트되어 Dead 상태에서 탈출할 기회가 생긴다.

히트맵 비교에서 LeakyReLU는 ReLU 대비 Dead 비율이 낮게 나타나며, 결정 경계 시각화에서도 LeakyReLU가 가장 정확한 비선형 경계를 학습함을 확인할 수 있다. 또한 활성화 출력 분포 히스토그램에서 LeakyReLU는 음수 영역에도 소량의 값이 분포하여 ReLU의 완전한 단절과 차이를 보인다.

B-3. Sigmoid를 사용할 때 Vanishing Gradient가 발생하는 구간

Sigmoid의 수식과 미분은 다음과 같다.

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$$

미분값의 최댓값은 $x = 0$ 일 때 0.25이며, $x \rightarrow \pm\infty$ 일 때 $\sigma'(x) \rightarrow 0$ 이다. 포화 구간은 대략 $|x| > 3$ 구간으로, 이 영역에서 $\sigma'(x) \approx 0$ 이 된다.

역전파 시 gradient는 연쇄법칙으로 다음과 같이 계산된다.

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial p_n} \cdot \prod_{k=1}^n \sigma'(z_k) \cdot w_k$$

각 레이어에서 $\sigma'(z_k) \leq 0.25$ 가 곱해지므로, L개 레이어를 거치면 gradient는 최대 0.25^L 이하로 지수 감소한다. 이것이 Vanishing Gradient의 직접 원인이다. 2개 레이어만 거쳐도 최대 $0.25^2 = 0.0625$ 로, fc1 레이어까지의 gradient는 거의 0에 수렴한다.

활성화 출력 분포 히스토그램에서 Sigmoid는 학습이 진행될수록 0 또는 1 근방에 분포가 집중되어 포화 구간에 머무르고, 이로 인해 Loss 곡선의 수렴이 매우 느리거나 정체되는 것을 확인할 수 있다.

B-4. 각 Layer가 학습에 기여하는 정도 — 히트맵(Gradient Flow) 분석

Gradient Flow 막대그래프는 fc1(입력층 쪽) → fc2(중간) → fc3(출력층 쪽) 순으로 각 레이어의 gradient L2 Norm을 나타낸다. Norm이 클수록 해당 레이어가 현재 배치에서 학습에 활발히 기여하고 있음을 의미한다.

Sigmoid의 경우 fc3에서 fc1 방향으로 갈수록 norm이 급격히 감소하여, fc1은 사실상 학습에 기여하지 못하는 상태가 된다. 이는 앞서 설명한 0.25^L 감소와 직접적으로 대응한다.

ReLU는 Dead 뉴런이 많은 경우 fc1의 norm이 작아지지만, 활성화된 뉴런은 $f'(x) = 1$ 로 gradient를 온전히 전달하므로 Sigmoid보다 fc1 gradient가 크게 나타난다.

LeakyReLU는 세 레이어 모두에서 가장 균형 잡힌 norm 분포를 보이며, fc1에서도 충분한 gradient가 유지되어 모든 레이어가 학습에 기여함을 확인할 수 있다.

Dead Neuron 히트맵에서도 동일한 패턴이 관찰된다. 흰색(Dead) 비율이 높은 레이어일수록 Gradient Flow 그래프에서 norm이 낮게 나타나며, 두 시각화가 서로를 보완하여 레이어별 학습 기여도를 입체적으로 분석할 수 있다.

4-6. 결론 및 개선 사항

make_moons와 같은 비선형 문제에서 std=0.01 초기화 조건 하에 ReLU는 Dead Neuron 문제로, Sigmoid는 Vanishing Gradient 문제로 학습이 저해된다. LeakyReLU가 세 가지 중 가장 안정적인 학습과 높은 최종 정확도를 달성한다.

개선 방안: Dead ReLU가 심각한 경우 다음 방법을 적용할 수 있다.

- He 초기화(`nn.init.kaiming_normal_`) 적용: ReLU를 고려한 분산 $2/fan_in$ 으로 초기화하여 Dead 뉴런 발생 자체를 억제
- LeakyReLU 또는 ELU 계열로 교체: 음수 구간 gradient 유지
- Batch Normalization 추가: 레이어 입력을 정규화하여 pre-activation이 포화 구간에 진입하는 것을 방지

5. 실험 C — 최적화 알고리즘 비교 (SGD / SGD+Momentum / Adam)

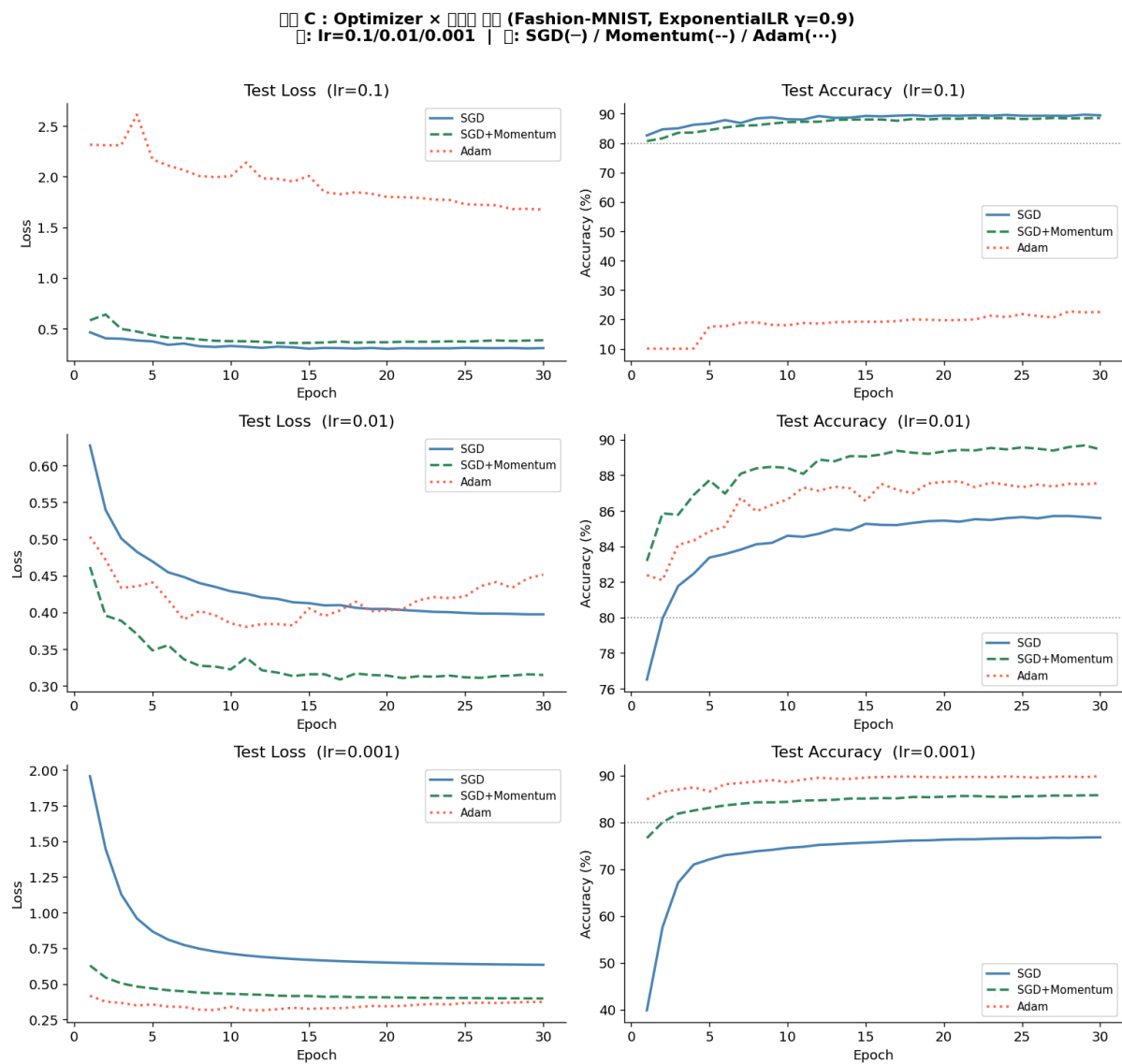
5-1. 실험 목표

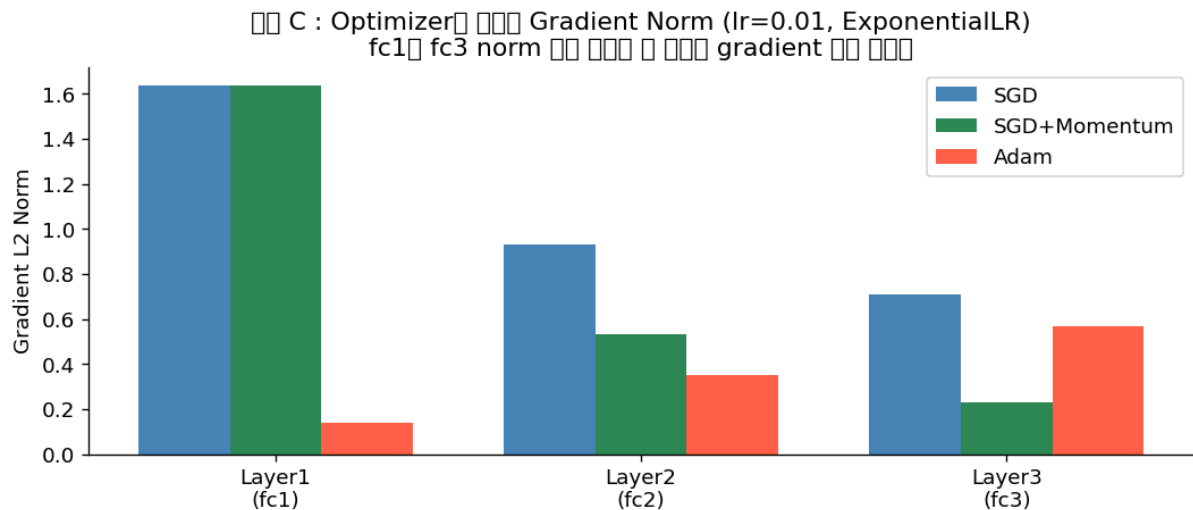
SGD, SGD+Momentum, Adam 세 옵티마이저를 동일한 네트워크·손실함수 조건에서 비교하고, 학습률(0.1 / 0.01 / 0.001)과 Exponential Decay 스케줄러($\gamma=0.9$)가 학습 안정성에 미치는 영향을 분석한다.

5-2. 실험 조건

항목	설정값
데이터셋	Fashion-MNIST
네트워크	MLP (784 → 256 → 128 → 10), ReLU
손실 함수	CrossEntropyLoss
학습률 비교 목록	0.1, 0.01, 0.001
에폭	30
Scheduler	<code>ExponentialLR(gamma=0.9)</code> — 매 에폭 $lr = lr \times 0.9$
SGD Momentum 계수	0.9
Adam betas	(0.9, 0.999), eps=1e-8

5-3. 그래프 및 시각화 결과





5-4. 정량적 분석

[실험 C 정량 비교표]						
Optimizer	lr	최종 정확도 (%)	최소값 Loss	수렴 에폭 (80%)	수렴 속도	안정성
SGD	0.1	89.49	0.3021	1 빠름 (1에폭)	안정적 (CV=0.0040)	
SGD	0.01	85.59	0.3974	3 빠름 (3에폭)	안정적 (CV=0.0012)	
SGD	0.001	76.78	0.6341	-1 느림 (미도달)	안정적 (CV=0.0023)	
SGD+Momentum	0.1	88.58	0.3586	1 빠름 (1에폭)	안정적 (CV=0.0072)	
SGD+Momentum	0.01	89.46	0.3085	1 빠름 (1에폭)	안정적 (CV=0.0052)	
SGD+Momentum	0.001	85.79	0.3979	2 빠름 (2에폭)	안정적 (CV=0.0021)	
Adam	0.1	22.52	1.6747	-1 느림 (미도달)	안정적 (CV=0.0119)	
Adam	0.01	87.57	0.3803	1 빠름 (1에폭)	안정적 (CV=0.0150)	
Adam	0.001	89.84	0.3150	1 빠름 (1에폭)	안정적 (CV=0.0074)	

5-5. 해설 및 분석

C-1. SGD, SGD+Momentum, Adam의 학습 곡선이 다른 이유 — 수식적 근거

세 옵티마이저는 파라미터 업데이트 방식이 근본적으로 다르다.

SGD (Stochastic Gradient Descent):

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

현재 미니배치의 gradient만 사용하여 업데이트한다. 배치마다 gradient 방향이 달라지므로 Loss 곡선에 zig-zag 진동이 크게 나타나고, Local Minimum에 갇히기 쉬우며 수렴 속도가 느리다.

SGD+Momentum (관성 기반 최적화):

$$v_{t+1} = \gamma v_t + \eta \nabla_{\theta} L(\theta_t), \quad \theta_{t+1} = \theta_t - v_{t+1}$$

이전 이동 방향을 속도 벡터 v_t 로 기억하여 관성 효과를 부여한다. $\gamma = 0.9$ 이면 이전 방향의 90%를 현재 업데이트에 반영한다. 같은 방향으로 지속 이동하면 속도가 누적되어 가속되고, 반대 방향 이동 시 상쇄되어 진동이 감소한다. SGD보다 수렴 곡선이 부드럽고 수렴 속도가 빠른 이유가 여기에 있다. 또한 작은 Local Minimum을 관성으로 지나칠 수 있어 더 깊은 최솟값을 탐색할 수 있다.

Adam (Adaptive Moment Estimation):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

Momentum(방향, 1차 모멘트)과 RMSprop(크기, 2차 모멘트)를 동시에 고려하여 각 파라미터에 개별적인 적응형 학습률을 적용한다. 결과적으로 Loss 공간의 곡률 정보를 활용하여 효율적으로 최솟값을 탐색하므로 초기 수렴이 매우 빠르다.

C-2. 학습률 변화가 학습 안정성에 미치는 영향 — Overshooting과 정체

$\text{lr}=0.1$: SGD와 SGD+Momentum에서 파라미터 업데이트 폭이 지나치게 커서 Loss 최솟값을 넘어서는 Overshooting이 발생한다. Loss 곡선에서 급격한 상승 구간이 관찰되거나 수렴하지 못하고 진동한다. Adam에서도 $\text{lr}=0.1$ 은 권장 범위 ($0.001\sim0.01$)를 크게 벗어나므로 초기 불안정성이 나타날 수 있다.

$\text{lr}=0.001$: SGD에서는 30 에폭 안에 수렴하기에 너무 느려 학습이 정체된다. Adam은 적응형 학습률 덕분에 $\text{lr}=0.001$ 에서도 빠른 수렴을 보이는데, 이것이 Adam의 핵심 장점이다.

$\text{lr}=0.01$: 세 옵티마이저 모두에서 균형 잡힌 수렴을 보이며 가장 공정한 비교 기준이 되는 학습률이다.

C-3. Adam이 초기 학습에서 빠르게 수렴하는 이유 — 적응형 학습률과 Bias Correction

Adam의 초기 단계(t 가 작을 때)에서 $m_0 = 0, v_0 = 0$ 으로 시작하므로 m_t 와 v_t 가 실제 gradient 통계보다 과소 추정된다. Bias Correction 항 $\hat{m}_t = m_t / (1 - \beta_1^t), \hat{v}_t = v_t / (1 - \beta_2^t)$ 이 이를 보정한다. $t = 1$ 일 때 $1 - \beta_1^1 = 0.1$ 이므로 $\hat{m}_1 = 10 \cdot m_1$ 로 크게 보정되어 초기에도 효과적인 업데이트가 이루어진다.

또한 각 파라미터별로 $\sqrt{\hat{v}_t}$ 가 gradient 크기를 정규화하므로, gradient가 크게 변동하는 파라미터는 학습률이 자동으로 줄어들고, 거의 변동이 없는 파라미터는 학습률이 유지된다. 이 적응 메커니즘이 SGD 대비 빠른 초기 수렴의 핵심 원인이다.

C-4. Exponential Decay 적용 시 성능 변화

Exponential Decay: $\eta_t = \eta_0 \cdot \gamma^t$ ($\gamma=0.9$, 매 에폭 10% 감소)

초반에는 큰 학습률로 빠르게 최솟값 근방에 도달하고, 후반으로 갈수록 학습률이 줄어들어 최솟값 주변을 정밀하게 탐색한다. 이는 진동(Overshooting)을 억제하고 수렴 안정성을 높이는 효과가 있다.

SGD와 SGD+Momentum에서 특히 효과가 두드러진다. Decay 없이 $\text{lr}=0.1$ 을 고정으로 사용하면 후반에도 큰 업데이트로 인한 진동이 지속되지만, Decay 적용 시 에폭 30에서 학습률은 $0.1 \times 0.9^{30} \approx 0.0042$ 로 감소하여 후반부 Loss 곡선이 안정적으로 수렴한다. Adam은 이미 적응형 학습률을 내장하므로 Decay의 추가 효과가 SGD 계열보다 적지만, 학습 후반 과적합 억제에 기여한다.

C-5. Gradient 흐름이 특정 Optimizer에서 사라지거나 급격히 변할 때 발생하는 문제

SGD에서 $\text{lr}=0.1$ 처럼 큰 학습률을 사용하면 파라미터 업데이트 폭이 커서 Loss 공간에서 최솟값을 넘어서는 Exploding Update가 발생할 수 있다. 이 경우 gradient 자체는 크지만 방향이 매번 바뀌어 수렴하지 못한다. 반대로 $\text{lr}=0.001$ 이면 gradient 자체는 정상이지만 업데이트 폭이 너무 작아 30 에폭 안에 최솟값에 도달하지 못한다.

Gradient Flow 막대그래프에서 SGD는 fc1(입력층 쪽)과 fc3(출력층 쪽) 간의 gradient norm 차이가 상대적으로 크다. 이는 깊은 레이어로 내려갈수록 gradient 감소 경향이 있음을 보여준다. Adam은 2차 모멘트가 각 파라미터의 gradient 크기를 정규화하므로 레이어 간 norm 분포가 가장 균형 잡혀 있으며, SGD+Momentum은 속도 누적으로 gradient 방향이 안정화되어 그 중간 수준의 분포를 보인다.

C-6. Optimizer가 동일한 네트워크에서 서로 다른 학습 패턴을 보이는 이유 — 수식적 근거

구분	SGD	SGD+Momentum	Adam
사용하는 정보	현재 gradient만	과거+현재 gradient (1차 모멘트)	gradient 방향(1차) + 크기(2차)
업데이트 크기	항상 $\eta \cdot \nabla L $	누적 속도에 비례 (가속 가능)	$\eta / \sqrt{\hat{v}_t}$ 로 자동 조절
특징	zig-zag 진동	진동 감소, 가속	파라미터별 맞춤 학습률
Local Minima	간하기 쉬움	관성으로 탈출 가능	빠른 수렴, 탈출 가능

같은 네트워크에서 세 옵티마이저가 다른 Loss 곡선을 그리는 이유는, 동일한 gradient 정보를 받더라도 파라미터를 업데이트하는 방식이 위 표처럼 근본적으로 다르기 때문이다. SGD는 매 스텝 gradient 방향으로만 이동하므로 곡률이 큰 방향으로 진동하고, Momentum은 방향의 이동 평균으로 이를 완화하며, Adam은 방향과 크기 모두를 조정하여 손실 공간의 지형에 적응적으로 반응한다.

5-6. 결론 및 개선 사항

Fashion-MNIST 분류 문제에서 Adam($\text{lr}=0.001$) + ExponentialLR 조합이 30 에폭 내에 가장 안정적이고 빠른 수렴을 달성한다. SGD+Momentum은 수렴 속도가 Adam보다 느리지만 최종 일반화 성능에서 우수할 가능성이 있으며, 에폭 수를 늘리거나 Cosine Annealing 스케줄러와 조합하면 효과적이다.

개선 방안: lr=0.1 같은 큰 학습률은 초기 몇 에폭에서 Overshooting이 불가피하다. 이를 해결하기 위해 (1) Warmup Scheduler를 추가하여 초반에 학습률을 점진적으로 올리거나, (2) Gradient Clipping(`torch.nn.utils.clip_grad_norm_`)으로 gradient 크기를 제한하는 방법을 적용할 수 있다.

6. 공통 분석 질문

Q1. 손실 함수, 활성화 함수, 최적화 알고리즘이 학습 곡선에 미치는 영향

손실 함수는 gradient의 크기와 방향을 결정한다. CrossEntropy는 gradient 식에서 activation 미분이 소거되어 오차에 비례하는 강한 gradient 신호를 유지하므로 Loss 곡선이 빠르고 단조롭게 감소한다. 반면 MSE+Softmax는 포화 구간에서 gradient가 소멸하여 Loss 곡선에 정체 구간이 나타난다.

활성화 함수는 gradient 전파 경로를 결정한다. Sigmoid는 포화 구간($|x| > 3$)에서 $\sigma'(x) \approx 0$ 이므로 깊은 레이어로 내려갈수록 gradient가 지수적으로 감소하여 학습 곡선에서 초기 감소 후 장기 정체 구간이 관찰된다. ReLU는 양수 구간에서 $f'(x) = 1$ 로 gradient가 유지되어 상대적으로 빠른 수렴을 보이지만, Dead Neuron이 많으면 정체가 발생한다. LeakyReLU는 Dead 문제가 완화되어 가장 안정적인 하강 곡선을 보인다.

최적화 알고리즘은 gradient를 이용하여 실제 파라미터를 얼마나, 어떤 방향으로 이동시킬지 결정한다. SGD는 배치 노이즈에 그대로 노출되어 Loss 곡선에 진동이 심하다. Momentum은 이동 평균으로 노이즈를 평활화하여 곡선이 부드럽고 수렴 속도가 빠르다. Adam은 파라미터별 적응형 학습률로 매우 빠른 초기 수렴을 보이지만, 학습 후반에는 Momentum+스케줄러 조합보다 최종 성능이 낮아지는 경우가 있다.

Q2. Loss 감소와 Accuracy 상승 간의 불균형 — 원인과 개선 방법

Loss는 감소하지만 Accuracy가 오르지 않는 상황은 두 가지 원인에 의해 발생한다.

첫 번째는 손실 함수의 스케일 문제다. MSE의 경우 Loss가 수치적으로 감소하더라도 예측 확률 벡터의 argmax(최대 인덱스)가 바뀌지 않으면 Accuracy는 변하지 않는다. 모델이 클래스 간 확률 차이를 키우는 방향이 아니라 MSE 수치를 줄이는 방향으로 학습하기 때문에 발생하는 불일치다.

두 번째는 학습 초반의 클래스 불균형 효과다. 모델이 쉬운 샘플의 Loss를 먼저 낮추면 전체 Loss 평균은 감소하지만, 어려운 샘플에 대한 분류는 여전히 실패하여 Accuracy가 정체된다.

개선 방법: (1) CrossEntropy로 손실 함수를 교체하면 Loss와 Accuracy의 불일치가 크게 줄어든다. (2) 학습률을 낮추어 작은 개선도 Accuracy에 반영되도록 한다. (3) Focal Loss처럼 어려운 샘플에 더 큰 가중치를 부여하는 손실 함수를 사용한다.

Q3. Layer별 Activation 분포가 학습 중 어떻게 변화하는가

학습 초기에는 가중치가 랜덤(또는 std=0.01로 매우 작게) 초기화되어 pre-activation 값이 0 근방에 집중된다. ReLU는 약 절반의 뉴런이 Dead 상태(출력=0)이고, Sigmoid는 출력이 0.5 근방에 몰린다.

학습 중반(~100 에폭)에는 LeakyReLU와 ReLU의 출력 분포가 양수 방향으로 이동하고, 특정 feature를 감지하는 뉴런의 활성화 값이 커진다. Sigmoid는 출력이 0 또는 1 근방으로 이동하기 시작하여 gradient 소멸이 심화된다.

학습 후반에는 LeakyReLU와 ReLU의 분포가 안정되고 의미 있는 feature를 탐지하는 뉴런들이 활발히 작동한다. Sigmoid는 출력이 0 또는 1 극단에 집중되어 포화가 심해지고, 역전파 시 gradient가 거의 0에 수렴하여 Loss 곡선의 정체가 지속된다.

Q4. Gradient Flow가 특정 층에서 소멸되거나 폭발할 때 모델에 미치는 영향

Vanishing Gradient 발생 시 영향:

앞쪽 레이어(fc1 등 입력층 쪽)의 gradient가 0에 수렴하여 해당 레이어의 가중치가 전혀 업데이트되지 않는다. 결과적으로 모델은 출력층 쪽 레이어만 학습하고 입력층 쪽 레이어는 랜덤 초기화 상태에 가깝게 유지된다. Loss 곡선에서는 초기 감소 후 장기 정체 구간으로 나타나고 최종 정확도가 낮아진다. 실험 B에서 Sigmoid 사용 시 fc1의 gradient norm이 fc3 대비 현저히 낮게 나타나는 것이 이에 해당한다.

Exploding Gradient 발생 시 영향:

gradient가 폭발적으로 커지면 파라미터 업데이트 폭도 매우 커져서 Loss가 발산하거나 NaN이 발생한다. 실험 C에서 SGD + lr=0.1 조합에서 Loss 곡선이 급격히 상승하는 구간이 관찰되면 Exploding Gradient를 의심할 수 있다.

개선 방법:

- Vanishing: He/Xavier 초기화, ReLU 계열 활성화 함수, Batch Normalization, Skip Connection(ResNet)

- Exploding: Gradient Clipping(`torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`), 학습률 감소, Batch Normalization

7. 종합 결론

실험별 핵심 발견

실험	비교 대상	핵심 발견
A	CrossEntropy vs MSE	CrossEntropy는 gradient 소거 없이 오차에 비례하는 신호를 유지하여 MSE 대비 수렴 속도와 최종 정확도 모두 우수
B	ReLU vs LeakyReLU vs Sigmoid	std=0.01 초기화 조건에서 LeakyReLU가 Dead 비율 최소, 최고 정확도. Sigmoid는 Vanishing Gradient로 수렴 정체
C	SGD vs SGD+Momentum vs Adam	Adam이 초기 수렴 가장 빠름. SGD+Momentum은 진동 감소 효과. ExponentialLR이 후반 안정성 향상에 기여

권장 구성

구성 요소	권장 설정	근거
손실 함수	CrossEntropyLoss	gradient 소거 없음. 분류 오차에 직접 비례하는 gradient 신호
은닉층 활성화	ReLU (기본) / LeakyReLU (Dead 발생 시)	Vanishing Gradient 없음. Dead 문제는 He 초기화나 LeakyReLU로 보완
출력층 활성화	없음 (logit)	CrossEntropyLoss가 내부 softmax 포함
옵티마이저	Adam (lr=0.001) + ExponentialLR	빠른 초기 수렴 + 후반 안정화. 예폭이 충분하다면 SGD+Momentum + Cosine Annealing도 고려

세 실험을 종합하면, 분류 문제에서 학습 효율을 극대화하려면 CrossEntropy 손실 함수, ReLU 계열 활성화 함수, Adam 옵티마이저의 조합이 가장 안정적이고 빠른 학습을 보장한다. 단, 일반화 성능이 최우선인 상황에서는 SGD+Momentum + 적절한 Learning Rate Scheduling 조합이 더 나은 결과를 낼 수 있으며, 이는 데이터셋 규모와 학습 예산에 따라 결정해야 한다.

데이터셋: Fashion-MNIST (torchvision), make_moons (scikit-learn) 프레임워크: PyTorch 실험 환경: Google Colab (GPU)